Prepared by
Sumit Mazumdar
Smart Contract Security Auditor

CONFIDENTIAL
For authorized recipients only

Audit findings are provided in good faith based on the code reviewed at the specified commit hash.

[H-1] Storing the password to on-chain makes it visible to everyone

Description: All data stored on the blockchain is visible to everyone, no matter the solidity visibility specifier. The `PasswordStore::s_password` variable is declared as private, but it is still visible to everyone. This is because the blockchain is a public ledger and anyone can view the contents of the blockchain.

We show one such method of reading any data off-chain below.

Impact: Any user can read the private password, severely breaking the functionality of the protocol.

Proof of Concept:

The below test case show how anyone can read the private password:

1. Create a locally running chain

```
make anvil
```

2. Deploy the contract

```
make deploy-password-store
```

3. Run the storage tool

We use `1` as the slot number because the `s_password` variable is the first variable in the contract, and the first variable is stored in the first slot.

```
cast storage show <contract-address> 1 --rpc-url http://127.0.0.1:8545
```

You'll get an output similar to the below one:

`0x6d7950617373776f726400000000000000000000000000000000000000000014`

You can parse then parse the hex to a string with: `cast parse-bytes32-string 0x6d7950617373776f726400000`

And you'll get the output:

`myPassword`

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password and store the encrypted password on-chain. This way, even if someone manages to read the encrypted password, they won't be able to decrypt it without the decryption key. However, you'd also likely want to remove the view function as you wouldn't want to send a transaction with the password that decrypts the password.

[H-2] Missing access control

Description: The `PasswordStore::setPassword` function is not protected by any access control mechanism. This means that any user can call this function and set a new password, even if they are not the owner of the contract. However, the function is intended to be used only by the `owner of the contract`.

```
1
2  function setPassword(string memory newPassword) external {
3  @>  // @audit missing access control
4      s_password = newPassword;
5      emit SetNewPassword();
6  }
```

Impact: Any user can set a new password, severely breaking the functionality of the protocol.

Proof of Concept: Add the following to the PasswordStore.t.sol file:

Code

```
1          function testFuzz_anyone_can_set_password(address randomAddress)
               public{
2           vm.assume(owner!=randomAddress);
3           vm.prank(randomAddress);
4           string memory expectedPassword = "myNewPassword";
5           passwordStore.setPassword(expectedPassword);
6
7           vm.prank(owner);
8           string memory actualPassword = passwordStore.getPassword();
9           assertEq(actualPassword, expectedPassword);
10      }
```

Recommended Mitigation: Add an access control mechanism to the `setPassword` function to ensure that only the owner can call this function.

```
1      if (msg.sender != s_owner) {
2          revert PasswordStore__NotOwner();
3      }
```

[I-1] The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist

Description: The `PasswordStore::getPassword` function is documented to take a parameter `newPassword`, but the function signature does not have any parameters.

```
1      /*
2       * @notice This allows only the owner to retrieve the password.
3  @>   * @param newPassword The new password to set.
```

```
     */
    function getPassword() external view returns (string memory) {
        if (msg.sender != s_owner) {
            revert PasswordStore__NotOwner();
        }
        return s_password;
    }
```

Impact: This is a documentation error and does not have any impact on the functionality of the contract.

Recommended Mitigation: Update the natspec to reflect the actual function signature.

```
- @param newPassword The new password to set.
```